모델 학습과 성능 평가

COW 데이터사이언스 트랙 6주차 이론 정리 · 김택동

---

1. 모델 학습이 뭘까 (손실 함수)
2. 경사하강법과 학습률, 옵티마이저
3. 에폭 · 배치 · 이터레이션, 학습 루프
4. 데이터 나누기, 과적합, 교차검증
5. 분류 모델 평가 (혼동행렬, 정밀도·재현율, ROC-AUC)
6. 회귀 모델 평가 (MAE, RMSE, R^2)
7. 객체 탐지 모델 평가 (IoU, mAP)
8. 평가할 때 조심할 것 / 정리

---

1. 모델 학습이 뭘까

지난주에 베이스라인 모델을 만들었다면, 이번엔 그 모델을 실제로 **학습**시키고 얼마나 잘하는지 **평가**하는 단계다.

모델 안에는 가중치(weight)라는 숫자들이 엄청 많이 들어 있다. 처음엔 이 숫자들이 거의 랜덤이라 모델이 엉뚱한 답을 낸다. 학습이란 결국 **모델이 낸 답과 실제 정답의 차이를 줄이는 방향으로 가중치를 조금씩 고치는 과정**이다.

여기서 "차이가 얼마나 나는지"를 숫자 하나로 계산해 주는 게 **손실 함수(loss function)**다. 손실이 크면 많이 틀린 거고, 작으면 잘 맞힌 거. 그래서 학습 = 손실을 최대한 작게 만드는 것이라고 보면 된다.

자주 쓰는 손실 함수 두 개

- **MSE (평균 제곱 오차)** : 숫자를 맞히는 회귀 문제에서 씀. (정답 − 예측)을 제곱해서 평균 낸 값. 제곱을 하니까 크게 틀린 걸 더 세게 혼낸다.
- **크로스엔트로피(Cross-Entropy)** : 종류를 맞히는 분류 문제에서 씀. 모델이 **정답 클래스에 준 확률**이 낮을수록 손실이 커진다.

> 크로스엔트로피 느낌 잡기 (정답이 "고양이"일 때, 손실 = −log(고양이에 준 확률))
> · 고양이일 확률 0.9라고 했으면 → 손실 약 0.11 (잘했음)
> · 0.5라고 했으면 → 약 0.69
> · 0.1이라고 했으면 → 약 2.30 (확신 없이 틀리면 손실이 확 커짐)

2. 경사하강법과 학습률

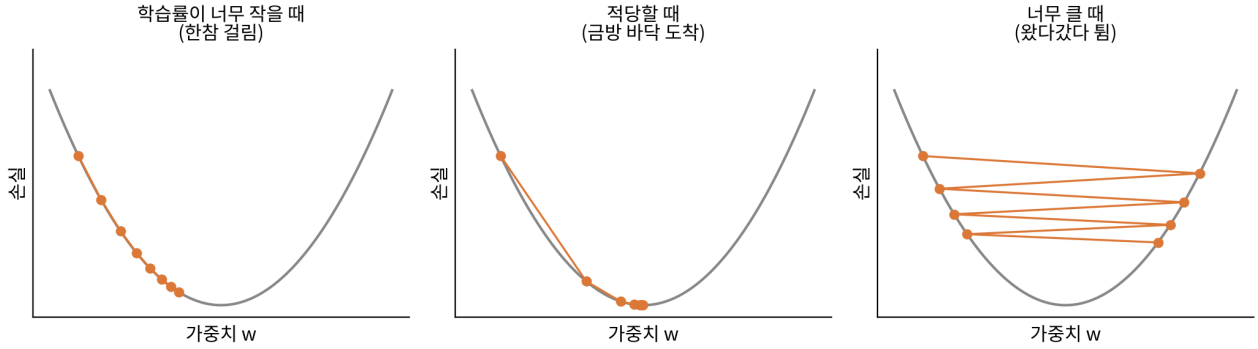
손실을 줄이려면 가중치를 어느 쪽으로 움직여야 할지 알아야 한다. 이걸 알려주는 게 **기울기(gradient)**다. 기울기는 "이 가중치를 살짝 키우면 손실이 늘어나나 줄어드나"를 알려 주는 값이라, **기울기 반대 방향**으로 가면

손실이 줄어든다.

산에서 눈 감고 내려갈 때 발로 경사를 더듬어서 낮은 쪽으로 한 발씩 내딛는 거랑 비슷하다. 이걸 **경사하강법(Gradient Descent)**이라고 한다.

```
새 가중치 = 지금 가중치 − 학습률 × 기울기
```

여기서 **학습률(learning rate)**은 한 번에 얼마나 크게 발을 내딛을지 정하는 값이다. 사람이 직접 정해 줘야 하는 값이라 하이퍼파라미터라고 부른다.



학습률에 따라 바닥(손실 최소)까지 가는 모습이 달라진다

- 너무 작으면: 방향은 맞는데 너무 찔끔찔끔 가서 학습이 오래 걸린다.
- 너무 크면: 바닥을 지나쳐서 반대편으로 튀고, 심하면 손실이 오히려 커지면서 발산한다.

옵티마이저

"어떻게 내려갈지"를 정하는 방법을 옵티마이저라고 부른다. 다 경사하강법을 조금씩 변형한 것들이다.

이름	한 줄 설명
SGD	데이터 일부만 보고 기울기를 구해서 바로 업데이트. 가장 기본.
SGD + 모멘텀	이전에 가던 방향을 관성처럼 조금 유지. 덜 흔들리고 더 빨리 내려감.
Adam	가중치마다 학습률을 알아서 조절해 줌. 설정 신경 덜 써도 웬만큼 잘 돼서 처음엔 보통 이걸 씀 (학습률 0.001이 흔한 기본값).

3. 에폭 · 배치 · 이터레이션

학습 로그를 보면 꼭 나오는 단어들인데 처음엔 헷갈려서 정리해 둠.

- **배치(batch)** : 한 번에 모델에 넣는 데이터 묶음. 데이터를 한꺼번에 다 넣으면 메모리가 터지니까 잘라서 넣는다. 묶음 크기를 배치 크기라고 함.
- **이터레이션(iteration)** : 배치 하나 넣고 가중치 한 번 업데이트하는 것 = 1 이터레이션.
- **에폭(epoch)** : 전체 데이터를 한 바퀴 다 본 것 = 1 에폭.

> 예시) 데이터 1,000장, 배치 크기 50
> → 1에폭 = 1,000 ÷ 50 = 20번 업데이트(이터레이션)
> → 10에폭 돌리면 총 200번 업데이트

배치 크기가 크면 한 번에 계산하는 양이 많아서 빠르지만 메모리를 많이 먹고, 작으면 업데이트가 자주 일어나는 대신 좀 들쭉날쭉하다. 보통 32, 64, 128 같은 값을 많이 쓴다.

학습 한 바퀴는 이렇게 돈다

① 배치를 모델에 넣어서 예측값을 뽑고(**순전파**) → ② 정답이랑 비교해서 손실을 계산하고 → ③ 손실이 각 가중치 때문에 얼마나 생겼는지 뒤에서부터 거꾸로 계산하고(**역전파**, 이게 기울기 구하는 과정) → ④ 옵티마이저가 가중치를 업데이트. 이걸 계속 반복한다. 파이토치로 쓰면 대충 이런 모양이다.

```
for epoch in range(10):                 # 전체 데이터를 10바퀴
    for x, y in train_loader:           # 배치 단위로 꺼내기
        pred = model(x)                 # ① 순전파
        loss = loss_fn(pred, y)         # ② 손실 계산
        optimizer.zero_grad()           #    지난번 기울기 지우기
        loss.backward()                 # ③ 역전파 (기울기 계산)
        optimizer.step()                # ④ 가중치 업데이트
```

매 에폭이 끝날 때마다 검증 데이터로 손실이랑 정확도를 찍어 보면서 학습이 잘 되고 있는지 확인한다. 이게 다음 장에서 볼 학습 곡선이다.

4. 데이터 나누기와 과적합

왜 데이터를 나눌까

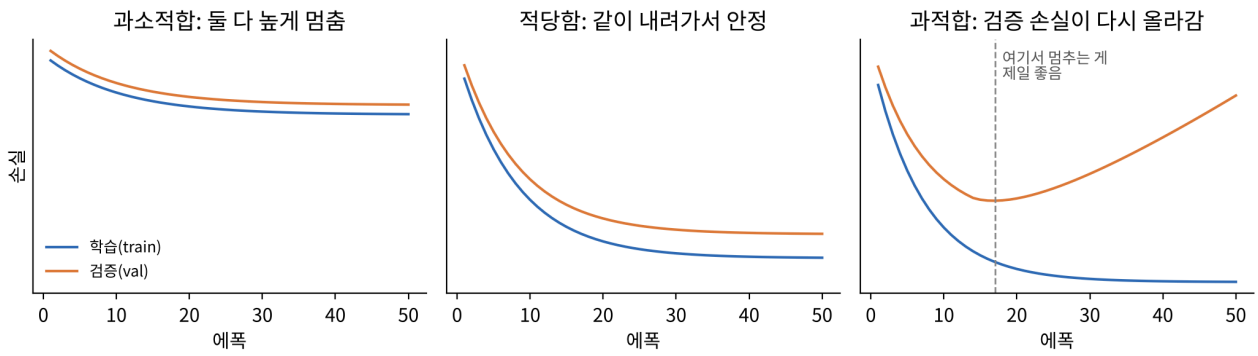
모델이 공부한 문제로 시험을 보면 당연히 점수가 잘 나온다. 그건 실력이 아니라 외운 거라서, 성능은 **모델이 한 번도 안 본 데이터**로 재야 한다. 그래서 데이터를 보통 세 덩어리로 나눈다.

구분	비유	하는 일
학습(train)	교과서·문제집	가중치를 실제로 업데이트하는 데 씀
검증(validation)	모의고사	학습 중간중간 성능 확인, 학습률 같은 설정 고르기, 언제 멈출지 정하기
테스트(test)	수능	모든 게 끝난 뒤 마지막에 딱 한 번 보는 최종 점수

비율은 7:1.5:1.5나 8:1:1 정도를 많이 쓴다. 클래스 비율이 한쪽으로 쏠린 데이터면 나눌 때도 비율이 유지되도록 **층화(stratify)** 옵션을 켜 주는 게 좋다.

과소적합과 과적합

학습 손실과 검증 손실을 에폭마다 그려 보면 모델 상태가 보인다.



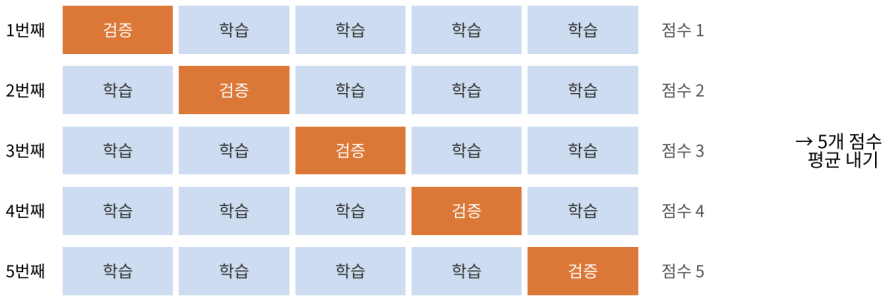
파란색 = 학습 손실, 주황색 = 검증 손실

- **과소적합(underfitting)** : 학습 데이터조차 잘 못 맞힘. 모델이 너무 단순하거나 학습을 덜 한 상태.
- **과적합(overfitting)** : 학습 데이터는 거의 다 맞히는데 새 데이터에선 못함. 문제집 답을 통째로 외운 상태. 학습 손실은 계속 내려가는데 검증 손실이 어느 순간부터 다시 올라가는 게 신호다.

과적합이 보이면 검증 손실이 제일 낮았던 지점에서 학습을 멈추는 방법이 있는데 이걸 **조기 종료(early stopping)**라고 한다. 보통 "검증 손실이 5에폭 동안 안 좋아지면 멈추고, 제일 좋았던 때의 가중치를 저장" 이런 식으로 쓴다. 과적합을 줄이는 다른 방법(드롭아웃, 데이터 증강 등)은 다음 주 성능 향상 주제에서 다룰 예정.

교차검증 (K-fold)

데이터가 적으면 검증 세트를 어떻게 떼느냐에 따라 점수가 운빨로 크게 달라진다. 그래서 데이터를 K조각으로 나누고, **조각을 돌아가면서 한 번씩 검증용으로 쓰고** 나머지로 학습해서 K번 점수를 낸 다음 평균을 낸다.



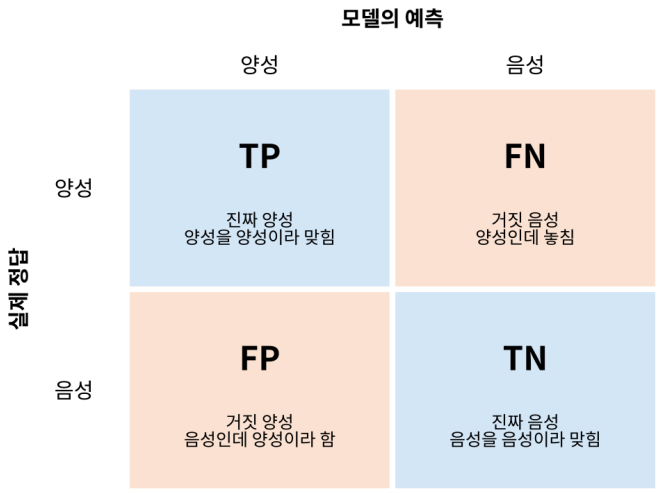
5-fold 교차검증. 모든 데이터가 한 번씩은 검증에 쓰인다

점수가 더 믿을 만해지는 대신 학습을 K번 해야 해서 시간이 K배 든다. 그래서 데이터 적은 머신러닝 모델에선 자주 쓰고, 학습 한 번에 몇 시간 걸리는 딥러닝에선 그냥 train/val/test로 나누는 경우가 많다.

5. 분류 모델 평가

혼동행렬부터

분류 지표는 거의 다 혼동행렬(confusion matrix)에서 나온다. 맞힌 것/틀린 것을 네 칸으로 나눈 표다. "양성"은 우리가 찾고 싶은 쪽(예: 환자, 스팸)이라고 생각하면 편하다.



T/F = 맞았나 틀렸나, P/N = 모델이 양성이라 했나 음성이라 했나

정확도, 정밀도, 재현율, F1

지표	계산	뜻
정확도 (Accuracy)	(TP+TN) / 전체	전체 중에 맞힌 비율
정밀도 (Precision)	TP / (TP+FP)	모델이 "양성!"이라고 한 것 중 진짜 양성 비율. 헛소리 안 하는 정도
재현율 (Recall)	TP / (TP+FN)	실제 양성 중에 모델이 찾아낸 비율. 안 놓치는 정도
F1 점수	2 × 정밀도 × 재현율 / (정밀도 + 재현율)	정밀도와 재현율의 조화평균. 둘 중 하나만 높으면 점수가 안 나옴

예시) 100명 검사, 실제 환자 10명. 모델 결과가 TP 8, FN 2, FP 6, TN 84 라면
· 정확도 = (8+84)/100 = **92%**
· 정밀도 = 8/(8+6) ≈ **0.57** → "환자"라고 한 14명 중 진짜는 8명
· 재현율 = 8/(8+2) = **0.80** → 환자 10명 중 8명 찾음
· F1 = 2×0.57×0.80/(0.57+0.80) ≈ **0.67**

정확도의 함정 : 위 데이터에서 모델이 아무 생각 없이 전부 "정상"이라고만 해도 정확도는 90%가 나온다. 근데 환자는 한 명도 못 찾음(재현율 0). 클래스가 한쪽에 몰려 있는 데이터에선 정확도만 보면 안 되고 정밀도·재현율·F1을 같이 봐야 한다.

정밀도 vs 재현율, 뭐가 더 중요할까

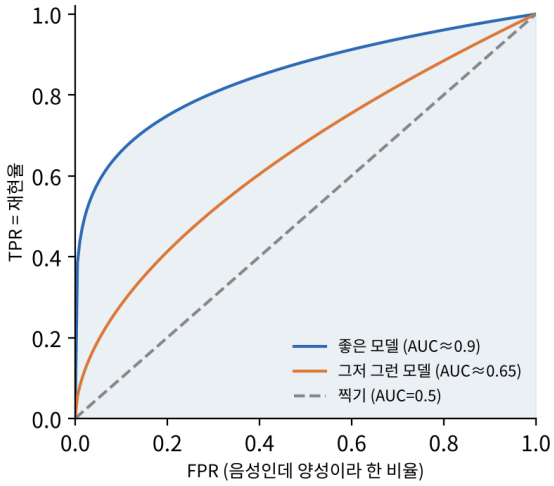
둘은 보통 한쪽을 올리면 다른 쪽이 떨어진다. 모델이 "확률 0.5 넘으면 양성"이라고 판단할 때 이 기준값(**임계값, threshold**)을 낮추면 양성을 더 많이 외쳐서 재현율은 오르고 정밀도는 떨어진다. 높이면 반대. 그래서 문

제 상황에 맞게 골라야 한다.

- **재현율이 중요한 경우** : 질병 검사, 불량품 검출, 사기 거래 탐지. 놓치면 큰일 나는 경우. 오진 좀 나도 일단 다 잡는 게 낫다.
- **정밀도가 중요한 경우** : 스팸 필터. 중요한 메일이 스팸함으로 가 버리면(FP) 그게 더 문제다.

ROC 곡선과 AUC

임계값 하나에서의 점수만 보면 모델 자체 실력인지 임계값을 잘 고른 건지 헷갈린다. 그래서 **임계값을 0부터 1까지 쭉 바꿔 가면서** 재현율(TPR)과 거짓 양성 비율(FPR)을 찍어서 이은 게 ROC 곡선이다.



왼쪽 위로 붙을수록 좋은 모델

곡선 아래 넓이가 **AUC**다. 1이면 완벽하게 구분, 0.5면 동전 던지기 수준. 임계값이랑 상관없이 "양성과 음성을 얼마나 잘 갈라 놓는지"를 숫자 하나로 볼 수 있어서 모델끼리 비교할 때 편하다.

클래스가 여러 개일 때

- **Top-1 / Top-5 정확도** : 이미지 분류처럼 클래스가 1,000개씩 되면, 모델이 1등으로 꼽은 답이 맞았는지(Top-1), 상위 5개 안에 정답이 있는지(Top-5)를 본다. ImageNet 성능 표에 항상 나오는 숫자.
- **macro 평균** : 클래스별로 정밀도·재현율을 구해서 그냥 평균. 데이터 적은 클래스도 똑같이 대접함.
- **weighted 평균** : 클래스별 데이터 개수만큼 가중치를 줘서 평균. scikit-learn의 `classification_report`를 쓰면 이게 다 한 번에 나온다.

6. 회귀 모델 평가

집값, 온도, 매출처럼 숫자를 예측하는 모델은 "얼마나 가깝게 맞혔나"를 본다.

지표	뜻	특징
MAE	|정답 − 예측|의 평균. 평균적으로 몇 만큼 틀렸나	해석이 제일 쉬움. 단위가 원래 데이터랑 같음
MSE	(정답 − 예측)²의 평균	크게 틀린 걸 엄청 세게 반영. 단위가 제곱이라 해석은 불편
RMSE	MSE에 루트 씌운 것	MSE 성질 유지 + 단위가 원래대로 돌아옴
R^2 (결정계수)	1 − (모델 오차 제곱합 / 그냥 평균으로 찍었을 때 오차 제곱합)	1에 가까울수록 좋음. 0이면 평균으로 찍는 거랑 같은 수준

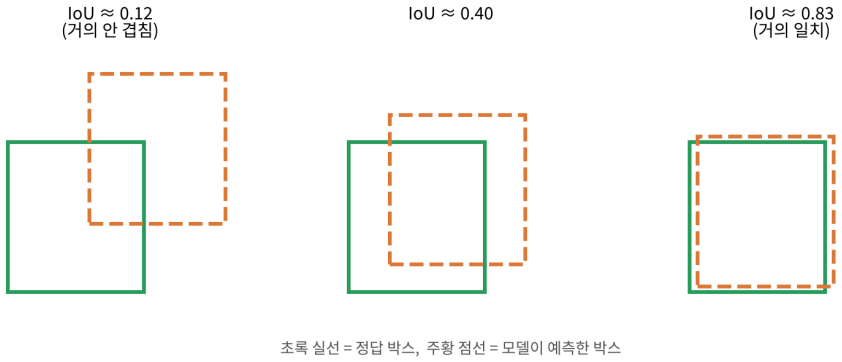
> 예시) 집값(억) 실제 3, 5, 7 / 예측 2.5, 5.5, 9 → 오차 −0.5, +0.5, +2
> · MAE = (0.5+0.5+2)/3 = **1.0**
> · MSE = (0.25+0.25+4)/3 = **1.5**, RMSE = $\sqrt{1.5} \approx$ **1.22**
> · R^2 : 평균(5)으로 찍었을 때 오차 제곱합 = 4+0+4 = 8, 모델은 4.5 → 1 − 4.5/8 ≈ **0.44**
> → 하나만 크게 틀렸는데 MAE보다 RMSE가 더 크게 나옴. RMSE가 큰 실수에 민감하다는 게 이런 뜻.

7. 객체 탐지 모델 평가

YOLO 같은 객체 탐지 모델은 "뭔지"만 맞히는 게 아니라 "어디 있는지" 박스도 그려야 해서 평가가 하나 더 붙는다.

IoU (Intersection over Union)

정답 박스와 예측 박스가 얼마나 겹치는지 보는 값이다. **겹친 넓이 ÷ 두 박스를 합친 넓이**. 0이면 전혀 안 겹침, 1이면 완전히 일치.



보통 IoU 0.5 이상이면 "제대로 찾았다"로 친다

AP와 mAP

IoU 기준을 넘고 클래스도 맞으면 TP, 아니면 FP로 센다. 그러면 분류처럼 정밀도와 재현율을 구할 수 있고, 모델의 확신도(confidence) 기준을 바꿔 가며 정밀도-재현율 곡선을 그릴 수 있다. 이 곡선 아래 넓이가

AP(Average Precision)이고, 클래스마다 AP를 구해서 평균 낸 게 **mAP**다.

- **mAP@0.5** : IoU 0.5 넘으면 맞은 걸로 쳐서 계산.
- **mAP@0.5:0.95** : IoU 기준을 0.5, 0.55, … 0.95까지 바꿔 가며 구한 값을 평균. 박스를 꼼꼼하게 그려야 점수가 나와서 훨씬 빡빡하다. COCO 데이터셋 기준 성능 표에 주로 나오는 게 이거.

8. 평가할 때 조심할 것

- **테스트 세트는 진짜 마지막에 한 번만.** 테스트 점수를 보고 설정을 고치고 다시 테스트하고를 반복하면 테스트 세트에 맞춰 버리는 거라 그 점수는 더 이상 객관적이지 않다. 고칠 땐 검증 세트를 보자.
- **데이터 누수 조심.** 정규화 같은 전처리를 전체 데이터로 먼저 하고 나누면 테스트 정보가 학습에 새어 들어간다. 나누고 나서 학습 데이터 기준으로 전처리해야 함. (3주차 전처리에서 본 내용)
- **기준점이랑 비교하기.** 정확도 85%가 좋은 건지 아닌지는 비교 대상이 있어야 안다. "제일 많은 클래스로 다 찍기"나 지난주 베이스라인 모델 점수를 옆에 같이 적어 두자.
- **한 번 돌린 결과만 믿지 않기.** 딥러닝은 초기 가중치나 데이터 섞는 순서에 따라 결과가 꽤 흔들린다. 시드를 고정하거나 몇 번 돌려서 평균을 보는 게 좋다.
- **목적에 맞는 지표 고르기.** 무작정 정확도가 아니라, 놓치는 게 문제인지(재현율) 잘못 외치는 게 문제인지(정밀도) 먼저 생각하고 지표를 정한다.

정리

상황	보는 것
학습이 잘 되고 있나	학습/검증 손실 곡선 → 과소적합·과적합 확인, 조기 종료
분류 (클래스 균형)	정확도 + 혼동행렬
분류 (클래스 불균형)	정밀도, 재현율, F1, ROC-AUC
클래스가 아주 많은 분류	Top-1 / Top-5 정확도, macro 평균 F1
회귀	MAE(해석용), RMSE(큰 오차 민감), R^2(설명력)
객체 탐지	IoU, mAP@0.5, mAP@0.5:0.95
데이터가 적을 때	K-fold 교차검증으로 점수 평균

한 줄로 줄이면, 학습은 **손실을 줄이는 방향으로 가중치를 조금씩 고치는 반복**이고, 평가는 **안 본 데이터에서 목적에 맞는 지표로 재는 것**이다. 다음 주에는 여기서 나온 점수를 실제로 끌어올리는 방법(하이퍼파라미터 튜닝, 정규화, 데이터 증강 등)을 정리할 예정.